



SECURITY AUDIT OF
BULLBIT



BULLBIT

Public Report

Version: 1.1

June 24, 2026

Verichains Lab

info@verichains.io

<https://www.verichains.io>

ABBREVIATIONS

Name	Description
Smart contract	A computer protocol intended to digitally facilitate, verify or enforce the negotiation or performance of a contract.
Ethereum	An open source platform based on blockchain technology to create and distribute smart contracts and decentralized applications.
Ether (ETH)	A cryptocurrency whose blockchain is generated by the Ethereum platform. Ether is used for payment of transactions and computing services in the Ethereum network.
Solidity	A contract-oriented, high-level language for implementing smart contracts for the Ethereum platform.
Solc	A compiler for Solidity.

EXECUTIVE SUMMARY

This Security Audit Report was prepared by Verichains Lab on *Jun 24, 2026*. We would like to thank the Bullbit for trusting Verichains Lab in auditing smart contracts. Delivering high-quality audits is always our top priority.

This audit focused on identifying security flaws in code and the design of the Bullbit. The scope of the audit is limited to the source code files provided to Verichains. Verichains Lab completed the assessment using manual, static, and dynamic analysis techniques.

During the audit process, the audit team had identified some vulnerable issues in the smart contract code.

TABLE OF CONTENTS

1. Management Summary	5
1.1. About Bullbit	5
1.2. About Verichains	5
1.3. Audit Methodology	6
1.4. Disclaimer	6
1.5. Acceptance Minute	7
2. Audit Overview	8
2.1. Audit Scope	8
2.2. Overview	10
3. Findings	12
3.1. HIGH - Unbonding Locks Continue Earning Unaccounted Rewards - FIXED	12
3.2. MEDIUM - Dust Funding Clears Emergency Unlock Without Solvency - FIXED	14
3.3. MEDIUM - Unbonding Duration Changes Retroactively Affect In-Progress Unbonding - FIXED	15
3.4. LOW - Governance Has No Direct ETH Withdraw Function - FIXED	16
3.5. LOW - Manual Reentrancy Guard Can Brick Entrypoints If Exit Is Missed - FIXED	17
3.6. LOW - Initialize Node Data Allows Silent Re-initialization of the Same NFT ID - ACKED	18
3.7. LOW - Burn For Migration Does Not Require Contract to Be Paused - FIXED	19
3.8. LOW - Tier Lock Duration Has No Minimum or Maximum Bound - ACKED	20
3.9. LOW - Unbonding Duration Has No Minimum Bound - FIXED	21
3.10. LOW - <code>getPendingReward()</code> Returns Stale Rewards - FIXED	22
3.11. LOW - <code>hValue</code> Can Be Set To Zero, Causing System-Wide Division by Zero Revert - ACKED	22
3.12. INFO - Node License NFT Is Pausable Despite Being Documented as Non-Pausable - ACKED	23
4. Version History	25

1. Management Summary

1.1. About Bullbit

Bullbit Protocol is a suite of smart contracts serving as the foundational infrastructure for the BUBI tokenomics ecosystem. Designed to be deployed on the Base L2 network, it facilitates a dynamic reward distribution and staking system.

The protocol features an ERC-721 Node License mechanism with customized transfer controls, an upgradeable Node Reward Vault for tracking off-chain generated reward metrics, and a fully separated Incentive Staking System that isolates user deposits from the protocol's reward budget to maintain fund integrity. Governed through a central administrative contract, Bullbit leverages the UUPS proxy pattern for secure protocol upgradeability.

1.2. About Verichains

Since 2017, Verichains has been a pioneer and leading blockchain security firm in APAC, with extensive expertise in security, cryptography, and core blockchain technology. Over 200 clients trust us to protect assets valued at more than \$50 billion, including several high-profile clients such as BNB Chain, Bybit, Aptos, Sui, Polygon, Klaytn, Wemix, Line Corp, Ronin Network, and Kyber Network.

With in-depth research and development of blockchain technology, Verichains offers a range of security services, including blockchain protocol and smart contract security audit, mobile application protection, key management solutions, on-chain risk monitoring, and red team/penetration testing services.

Verichains is a certified CREST penetration testing member, delivering professional testing services at the highest global standards.



Homepage: <https://www.verichains.io>

Blog: <https://blog.verichains.io>

Email: info@verichains.io

Twitter: <https://twitter.com/Verichains>

Facebook: <https://facebook.com/verichains>

Linkedin: <https://www.linkedin.com/company/verichains/>

1.3. Audit Methodology

Our security audit process for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using public and RK87, our in-house smart contract security analysis tool.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that were considered during the audit of the smart contract:

- Integer Overflow and Underflow
- Timestamp Dependence
- Race Conditions
- Transaction-Ordering Dependence
- DoS with (Unexpected) revert
- DoS with Block Gas Limit
- Gas Usage, Gas Limit and Loops
- Redundant fallback function
- Unsafe type Inference
- Reentrancy
- Explicit visibility of functions state variables (external, internal, private and public)
- Logic Flaws

For vulnerabilities, we categorize the findings into categories as listed in table below, depending on their severity level:

SEVERITY LEVEL	DESCRIPTION
CRITICAL	A vulnerability that can disrupt the contract functioning; creates a critical risk to the contract; required to be fixed immediately.
HIGH	A vulnerability that could affect the desired outcome of executing the contract with high impact; needs to be fixed with high priority.
MEDIUM	A vulnerability that could affect the desired outcome of executing the contract with medium impact in a specific scenario; needs to be fixed.
LOW	An issue that does not have a significant impact, can be considered as less important.

Table 1: Severity levels

1.4. Disclaimer

Bullbit acknowledges that the security services provided by Verichains, are conducted to the best of their professional abilities but cannot guarantee 100% coverage of all security vulnerabilities. Bullbit

understands and accepts that despite rigorous auditing, certain vulnerabilities may remain undetected. Therefore, Bullbit agrees that Verichains shall not be held responsible or liable, and shall not be charged for any hacking incidents that occur due to security vulnerabilities not identified during the audit process.

1.5. Acceptance Minute

This final report served by Verichains to the Bullbit will be considered an Acceptance Minute. Within 7 days, if no any further responses or reports is received from the Bullbit, the final report will be considered fully accepted by the Bullbit without the signature.

2. Audit Overview

2.1. Audit Scope

This audit focused on identifying security flaws in code and the design of the Bullbit. It was conducted on three private git repositories:

- **Bullbit-Governance-Smart-Contract**
- **Bullbit-Staking-Smart-Contract**
- **Bullbit-Node-Reward-Smart-Contract**

The latest hash of each repo as below:

Bullbit-Governance-Smart-Contract

SHA256 Sum	File
12aacb8415158674b1414856428580f8e9105d41bf3b62efc46b0dd05139fbbc	<code>contracts/Governance.sol</code>
97a1ab9872015aa6b4f81480cff94199d7668a35c958e814a20fc61b80780104	<code>contracts/interfaces/IAdminable.sol</code>
09c521441c751bac8af070bb8290ab7baeb6a5562503569dd9888d1aeceb74d5	<code>contracts/interfaces/IBurnForMigration.sol</code>
f434c10b6ff6713273961edb5bf63e4a4c48aa00bb34d8c93eacc650103536d6	<code>contracts/interfaces/IGovernable.sol</code>
66950dba4eeddaf51570177044bf170c83e0c03845b63ebf41bb52ba06da5a94	<code>contracts/interfaces/IOperatorable.sol</code>
c278ba8ef68f19763ec7a73676ed5cb7ab7b5dd70900166aca498178277bcfcd	<code>contracts/interfaces/IOwnable2StepLike.sol</code>
b4078b6c9ff525f62bca8e125ec3b6c735cdbf16356c571c50c5199ee117f31d	<code>contracts/interfaces/IPausableLike.sol</code>
cd8ba99748991bad0fc292c9c8a602053f0802c99006dee8ea7208e255547a9a	<code>contracts/interfaces/IRescueable.sol</code>
cdcbc8aca9229f90c2f7e1d73ec6ef79563c4dfd80b063096a7111dabf5297123	<code>contracts/ interfaces/IUUPSUpgradeableLike.sol</code>
7d2336ed5a491533f2683a850d5ff62649e46b2086108f775faa2dcdb1600357	<code>contracts/interfaces/targets/ IIncentiveRewardVaultGovernanceTarget.sol</code>
8402575102337bb3b901b22a080a2866b19716cf6083a4a8719ea3e994644fb5	<code>contracts/interfaces/targets/ IIncentiveStakingVaultGovernanceTarget.sol</code>
b5b304db9c89de2171bbe662817d1d6c59128b6e5d6e7af0025ff8e46df1e5bd	<code>contracts/interfaces/ targets/INodeLicenseNFTGovernanceTarget.sol</code>

98afdb31bdfe890f9f3afac9d380548a f42c84107a27ba933155edbea46a3fba	contracts/interfaces/ targets/INodeRewardVaultGovernanceTarget.sol
7d32762726ea9a491aba1513939506a4 1c0589ade82943c4f751e342c3e56e46	contracts/interfaces/ targets/IVestingGovernanceTarget.sol

Bullbit-Staking-Smart-Contract

SHA256 Sum	File
8b8f59227becd2601848c0a843dc14b0 a92e177d769996ac3d3a74d70a0e539a	contracts/IncentiveRewardVault.sol
f0a4ad9a3b89943e7d092de155336c42 ad367842a33e7e8617b66d762d8dbad1	contracts/IncentiveStakingVault.sol
8ae3073c55730595b34996c7da5f62a4 0e0551872bf5517269e895c847192f	contracts/proxy/BullbitERC1967Proxy.sol
97a1ab9872015aa6b4f81480cff94199 d7668a35c958e814a20fc61b80780104	contracts/interfaces/IAdminable.sol
09c521441c751bac8af070bb8290ab7b aeb6a5562503569dd9888d1aeceb74d5	contracts/interfaces/IBurnForMigration.sol
971d12ea70f449af2000bcb5e5a6e9db 45fde558fd9ea78bf3776f0caed26c75	contracts/interfaces/IERC20Burnable.sol
f434c10b6ff6713273961edb5bf63e4a 4c48aa00bb34d8c93eacc650103536d6	contracts/interfaces/IGovernable.sol
de18e2fdcc94c34fa86dce5a890b945e 8da528fc0a8ea1e0291fe88e7581b7c5	contracts/ interfaces/IIncentiveRewardVault.sol
29f09726a1116c027738ec83339f88c8 8958dabde02d07ded2e02e04159d76e0	contracts/ interfaces/IIncentiveStakingVault.sol
cd8ba99748991bad0fc292c9c8a60205 3f0802c99006dee8ea7208e255547a9a	contracts/interfaces/IRescueable.sol

Bullbit-Node-Reward-Smart-Contract

SHA256 Sum	File
ca5da3c2d8b56457aed4b7472ab510c9 6ff74b5e03fb194034cbd30cbbf95322	contracts/NodeLicenseNFT.sol
3dccb9282a6ce2a1fb858d915376c24 379bb45b313d49f56b5cd18ebc7a615f	contracts/NodeRewardVault.sol
f40775a5f45431646c1ad1bf84402c7c dccc0e7c22a040437ce5a539b67d8b6c	contracts/proxy/BullbitERC1967Proxy.sol

97a1ab9872015aa6b4f81480cff94199 d7668a35c958e814a20fc61b80780104	contracts/interfaces/IAdminable.sol
09c521441c751bac8af070bb8290ab7b aeb6a5562503569dd9888d1aeceb74d5	contracts/interfaces/IBurnForMigration.sol
f434c10b6ff6713273961edb5bf63e4a 4c48aa00bb34d8c93eacc650103536d6	contracts/interfaces/IGovernable.sol
66950dba4eeddaf51570177044bf170c 83e0c03845b63ebf41bb52ba06da5a94	contracts/interfaces/IOperatorable.sol
b4078b6c9ff525f62bca8e125ec3b6c7 35cdbf16356c571c50c5199ee117f31d	contracts/interfaces/IPausableLike.sol
cd8ba99748991bad0fc292c9c8a60205 3f0802c99006dee8ea7208e255547a9a	contracts/interfaces/IRescueable.sol

2.2. Overview

The Bullbit Protocol consists of a suite of smart contracts written in Solidity (0.8.30) and built on top of **OpenZeppelin Contracts**. The system primarily leverages the **UUPS proxy pattern** for upgradeability, ensuring the protocol can evolve securely over time.

Governance Manager (**Bullbit-Governance-Smart-Contract**)

Serving as the central administrative controller, **Governance.sol** manages the BUBI tokenomics ecosystem. Rather than storing a list of downstream addresses, the contract requires targets to be passed dynamically during function calls, verifying that each target is a valid, deployed contract. To provide clear and safe administration, all standard operations are handled through explicitly defined functions, such as `updateStakingTierConfig` and `setUnbondingDuration`. For more complex or unforeseen needs, a fallback `executeGenericTransaction()` is available, though it is strictly restricted to the owner. This governance contract is fully UUPS upgradeable.

Node License Ecosystem (**Bullbit-Node-Reward-Smart-Contract**)

The **NodeLicenseNFT.sol** contract provides an ERC-721 implementation with customized transfer logic. By utilizing a `transferEnabled` flag, the protocol can prevent regular token transfers and approvals when disabled, ensuring that tokens remain locked unless explicitly allowed. Minting operations are also tightly controlled, restricted solely to the admin role. Notably, this NFT contract is designed to be **not upgradeable**, providing a permanent foundation for node identities.

Working alongside the NFT is the **NodeRewardVault.sol**, an upgradeable ledger that tracks node rewards. A designated operator is responsible for injecting initial data and providing ongoing updates. To maintain strict accounting, the contract enforces on-chain invariants: the sum of locked and unlocked balances must always match the total allocated amount, and unlocked balances can only ever increase. When users claim their rewards, the vault verifies their NFT ownership in real-time before releasing the funds.

Incentive Staking System (**Bullbit-Staking-Smart-Contract**)

The staking mechanics are split across two upgradeable contracts to ensure a clear separation of funds: **IncentiveStakingVault.sol** secures user deposits, while **IncentiveRewardVault.sol** manages the reward budget, disbursing funds only when instructed by the staking vault.

At the core of the reward distribution is the `updateGlobalIndex()` function, which runs before any state-altering transaction. It calculates the reward debt for each tier and updates a global index using the following formula:

```
APR          = (kValue × 1018) / sqrt(hValue + TotalStakedGlobal / 1018)  
  
DeltaIndex   = (APR × Δt) / SECONDS_PER_YEAR  
NewDebt      = (TierStake × DeltaIndex) / 1018
```

To protect users, the system includes a built-in emergency safeguard. If the unallocated reward budget ever falls below the newly generated debt, the `isEmergencyUnlockActive` flag is automatically triggered. Once active, new deposits are reverted, and the system allows bypassing any remaining lock periods. This emergency state requires manual intervention from the owner to reset.

Note that, the `initiateUnstake()` function is intentionally permissionless. This design allows anyone (such as automated keepers or the Admin) to trigger the unbonding process on behalf of any user, provided the user's staking lock duration has expired or the emergency unlock flag is active.

Additionally, the protocol enforces an unbonding duration (configurable by the owner up to a maximum of 30 days). This duration is snapshotted the moment the unstaking process begins, ensuring that any future parameter changes won't affect users who are already in the process of withdrawing. Finally, to facilitate future transitions, a `burnForMigration()` function is available when the contract is paused.

3. Findings

#	Title	Severity	Status
1	Unbonding Locks Continue Earning Unaccounted Rewards	HIGH	Fixed
2	Dust Funding Clears Emergency Unlock Without Solvency	MEDIUM	Fixed
3	Unbonding Duration Changes Retroactively Affect In-Progress Unbonding	MEDIUM	Fixed
4	Governance Has No Direct ETH Withdraw Function	LOW	Fixed
5	Manual Reentrancy Guard Can Brick Entrypoints If Exit Is Missed	LOW	Fixed
6	Initialize Node Data Allows Silent Re-initialization of the Same NFT ID	LOW	Acknowledged
7	Burn For Migration Does Not Require Contract to Be Paused	LOW	Fixed
8	Tier Lock Duration Has No Minimum or Maximum Bound	LOW	Acknowledged
9	Unbonding Duration Has No Minimum Bound	LOW	Fixed
10	<code>getPendingReward()</code> Returns Stale Rewards	LOW	Fixed
11	<code>hValue</code> Can Be Set To Zero, Causing System-Wide Division by Zero Revert	LOW	Acknowledged
12	Node License NFT Is Pausable Despite Being Documented as Non-Pausable	INFO	Acknowledged

3.1. HIGH - Unbonding Locks Continue Earning Unaccounted Rewards - FIXED

Position

- `Bullbit-Staking-Smart-Contract/contracts/IncentiveStakingVault.sol`

Description

`initiateUnstake()` removes a lock from `totalStakedGlobal` and `totalStakedTier` but leaves `amount` and `entryIndex` live, so `claimReward()` can later pay the unbonding lock from index growth that was never charged to `unallocatedBudget`.

After a lock enters unbonding, it is no longer included in `totalStakedTier[tierId]`, so future `updateGlobalIndex()` calls only charge reward debt for still-active stake. However, the unbonding lock can still call `claimReward()`, and `_claimReward()` continues calculating rewards from `record.amount` and the tier's updated `globalIndices`.

This lets an unbonding lock receive rewards that were not included in the debt deducted from **unallo**
catedBudget.

```
function initiateUnstake(uint256 lockId) external returns (uint256 reward) {
    ...
    reward = _claimReward(record, lockId);

    uint256 tierId = record.tierId;
    uint256 amount = record.amount;
    totalStakedGlobal -= amount;
    totalStakedTier[tierId] -= amount;
    record.unbondingStartTimestamp = block.timestamp;
    ...
}

function claimReward(uint256 lockId) external returns (uint256 reward) {
    _requireNotPaused();
    _requireNotMigrated();
    _enterNonReentrant();

    LockupRecord storage record = _requireLockup(lockId);
    if (record.userAddress != msg.sender) revert UnauthorizedLockupCaller(lockId,
msg.sender);

    reward = _claimReward(record, lockId);
    _exitNonReentrant();
}

function _claimReward(LockupRecord storage record, uint256 lockId) internal returns
(uint256 reward) {
    if (address(incentiveRewardVault) == address(0)) revert RewardVaultNotSet();

    updateGlobalIndex();

    uint256 currentIndex = globalIndices[record.tierId];
    reward = Math.mulDiv(record.amount, currentIndex - record.entryIndex, PRECISION);
    record.entryIndex = currentIndex;

    if (reward > 0) {
        incentiveRewardVault.transferReward(record.userAddress, reward);
    }

    emit RewardClaimed(record.userAddress, lockId, reward);
}
```

Recommendation

Stop unbonding locks from accruing or claiming future rewards after they are removed from tier totals. Either reject reward claims for records with **unbondingStartTimestamp != 0**, or keep unbonding amounts in reward debt accounting until withdrawal.

Update

• **Jun 19, 2026:** The issue has been acknowledged and fixed by the development team in commit **3c74fdf7225244683e9364e8b31d75814ee9b3e0**.

3.2. MEDIUM - Dust Funding Clears Emergency Unlock Without Solvency - FIXED

Position

- Bullbit-Staking-Smart-Contract/contracts/IncentiveRewardVault.sol
- Bullbit-Staking-Smart-Contract/contracts/IncentiveStakingVault.sol

Description

`addRewardBudget()` unconditionally clears `isEmergencyUnlockActive` after any positive reward top-up, without checking whether the new budget is sufficient to cover the outstanding reward debt that caused emergency mode.

This creates a dust-funding path. If the staking vault is already in emergency mode because accrued rewards exceeded `unallocatedBudget`, any account can call `fundRewardPool(1)` through the reward vault. `fundRewardPool()` calls `updateGlobalIndex()` first, but `updateGlobalIndex()` immediately returns while emergency mode is active and advances `lastUpdateTimestamp` without accounting for the unpaid reward debt. Then `addRewardBudget(1)` adds only the dust amount and sets `isEmergencyUnlockActive = false`.

As a result, emergency unlock can be disabled even though the protocol is still underfunded. Locked users who were relying on emergency mode to exit can be forced back into the normal lock schedule until insolvency is detected again.

```
// IncentiveRewardVault.sol
function fundRewardPool(uint256 amount) external {
    ...
    bubiToken.safeTransferFrom(msg.sender, address(this), amount);
    incentiveStakingVault.updateGlobalIndex();
    incentiveStakingVault.addRewardBudget(amount);
}

// IncentiveStakingVault.sol
function updateGlobalIndex() public {
    ...
    if (totalStakedGlobal == 0 || isEmergencyUnlockActive) {
        lastUpdateTimestamp = currentTimestamp;
        emit GlobalIndexUpdated(currentTimestamp, 0, unallocatedBudget);
        return;
    }
    ...
}

function addRewardBudget(uint256 amount) external {
    _requireRewardVault();
    _requireNotMigrated();
    if (amount == 0) revert ZeroAmount();

    unallocatedBudget += amount;
    isEmergencyUnlockActive = false;
}
```

Recommendation

Do not clear emergency mode merely because a positive amount was funded. Emergency mode should only be cleared after the staking vault recomputes accrued debt and proves the post-funding budget is sufficient. Also avoid advancing `lastUpdateTimestamp` while emergency mode is active unless the unpaid interval is actually accounted for.

Update

• **Jun 19, 2026:** The issue has been acknowledged and fixed by the development team in commit `cedceb6337081e37ada86d64c54dad500ba8`. The `isEmergencyUnlockActive` flag is no longer in `addRewardBudget` function.

3.3. MEDIUM - Unbonding Duration Changes Retroactively Affect In-Progress Unbonding - FIXED

Position

- `Bullbit-Governance-Smart-Contract/contracts/Governance.sol`
- `Bullbit-Staking-Smart-Contract/contracts/IncentiveStakingVault.sol`

Description

`withdraw()` reads the global `unbondingDuration` at call time, not the value active when `initiateUnstake()` was called. When Governance calls `updateTargetUnbondingDuration()` after users have started unbonding, their withdrawal deadlines are changed retroactively.

If the duration is increased, a user who initiated unbonding expecting a 7-day wait can be forced to wait 30 days or more. In the extreme case, Governance can delay all in-progress withdrawals indefinitely by repeatedly increasing the duration.

```
// Governance.sol
function updateTargetUnbondingDuration(address target, uint256 newDuration) external
onlyOwner {
    _callTarget(
        target,
        0,
        abi.encodeCall(IIncentiveStakingVaultGovernanceTarget.setUnbondingDuration,
            (newDuration))
    );
    emit TargetUnbondingDurationUpdated(msg.sender, target, newDuration);
}

// IncentiveStakingVault.sol
function setUnbondingDuration(uint256 newDuration) external {
    _checkOwner();
    if (newDuration == 0) revert ZeroAmount();

    uint256 previousDuration = unbondingDuration;
    unbondingDuration = newDuration;
    emit UnbondingDurationUpdated(previousDuration, newDuration);
}

function withdraw(uint256 lockId) external {
    ...
    uint256 withdrawableAt = record.unbondingStartTimestamp + unbondingDuration;
```

```
    if (block.timestamp < withdrawableAt) revert UnbondingPeriodNotMet(lockId,
withdrawableAt);
    ...
}
```

Recommendation

Snapshot the applicable unbonding duration in each lock record when unbonding starts, and use that per-lock value in `withdraw()`. If global changes should apply to existing unbonding positions, restrict increases or apply them only after a timelock so users can react.

Update

• **Jun 19, 2026:** The issue has been acknowledged and fixed by the development team in commit `cddacedceb6337081e37ada86d64c54dad500ba8`. The `IncentiveStakingVault` snapshotted the `withdrawableAtTimestamp` for user when the user initiates the unbonding period.

3.4. LOW - Governance Has No Direct ETH Withdraw Function - FIXED

Position

• `Bullbit-Governance-Smart-Contract/contracts/Governance.sol`

Description

`Governance` accepts ETH via a bare `receive()` function but has no direct function to withdraw ETH held by `Governance` itself. `rescueTargetETH()` sends ETH from a target contract, not from `Governance`.

This is not a permanent loss of funds because the owner can use `executeGenericTransaction()` to forward ETH from `Governance` to a contract target, and that target can then expose its own withdrawal path. However, the recovery path is indirect, requires a helper contract, and cannot send directly to an EOA because `_validateTarget()` rejects targets with no code.

```
receive() external payable {}

function rescueTargetETH(address target, address to, uint256 amount) external onlyOwner {
    _requireNonZero(to);
    _callTarget(target, 0, abi.encodeCall(IRescuable.rescueETH, (to, amount)));
    emit ETHRescuedFromTarget(msg.sender, target, to, amount);
}

function executeGenericTransaction(address target, uint256 value, bytes calldata data)
external payable onlyOwner {
    bytes4 selector = _selectorOf(data);
    _callTarget(target, value, data);
    emit GenericTransactionExecuted(msg.sender, target, value, selector, data);
}

function _validateTarget(address target) internal view {
    _requireNonZero(target);
}
```



```
if (target == address(this) || target.code.length == 0) {  
    revert InvalidTarget(target);  
}  
}
```

Recommendation

Add a direct owner-only ETH rescue function, or remove `receive()` if **Governance** is not intended to hold ETH. A direct rescue path avoids requiring a helper contract for accidental ETH recovery.

Update

• **Jun 19, 2026:** The issue has been acknowledged and fixed by the development team. The **Governance** contract has a new `rescueETH` function to withdraw ETH held by the owner.

3.5. **LOW** - Manual Reentrancy Guard Can Brick Entrypoints If Exit Is Missed - **FIXED**

Position

- `Bullbit-Staking-Smart-Contract/contracts/IncentiveStakingVault.sol`
- `Bullbit-Staking-Smart-Contract/contracts/IncentiveRewardVault.sol`

Description

The staking and reward vaults implement reentrancy protection through manual `_enterNonReentrant()` and `_exitNonReentrant()` calls instead of a modifier. This pattern is easy to misuse because every protected function must remember to call `_exitNonReentrant()` on every non-reverting success path.

Current protected functions appear to call `_exitNonReentrant()` on their explicit success paths. However, if a future function calls `_enterNonReentrant()` and returns early without calling `_exitNonReentrant()`, `_reentrancyStatus` remains 2. After that, all protected functions will revert with `ReentrancyDetected()`, effectively bricking those entrypoints until an upgrade fixes the storage value.

```
function _enterNonReentrant() internal {  
    if (_reentrancyStatus == 2) revert ReentrancyDetected();  
    _reentrancyStatus = 2;  
}  
  
function _exitNonReentrant() internal {  
    _reentrancyStatus = 1;  
}
```

Recommendation

Replace the manual enter/exit calls with a `nonReentrant` modifier so the reset is structurally tied to the function body.

Update

• **Jun 19, 2026:** The issue has been acknowledged and fixed by the development team. Both contracts was updated to use [ReentrancyGuard](#).

3.6. **LOW** - Initialize Node Data Allows Silent Re-initialization of the Same NFT ID - **ACKED**

Position

• [Bullbit-Node-Reward-Smart-Contract/contracts/NodeRewardVault.sol](#)

Description

The `whenNotInitialized` modifier gates on the global `isInitialized` flag, which is only set to `true` by `startRewardDistribution()`. Before `startRewardDistribution()` is called, `initializeNodeData()` can be invoked arbitrarily many times. Within a single call or across multiple calls, the same `nftId` can appear more than once, and the later write silently overwrites the earlier one.

This creates two risks. First, large off-chain batches can contain duplicate entries for the same NFT, and a typo or tooling bug silently applies the second value. Second, a malicious or compromised operator can change `claimedBalance`, inflate `totalAllocated`, or modify `unlockedBalance` before reward distribution starts.

```
function initializeNodeData(InitPayload[] calldata payloads)
    external
    onlyOperator
    whenNotPaused
    whenNotMigrated
    whenNotInitialized
{
    uint256 length = payloads.length;
    if (length == 0) revert EmptyPayloads();

    for (uint256 i = 0; i < length; ++i) {
        InitPayload calldata payload = payloads[i];
        _requireExistingNodeLicense(payload.nftId);
        ...
        NodeData storage node = nodes[payload.nftId];
        node.totalAllocated = payload.totalAllocated;
        node.lockedBalance = payload.lockedBalance;
        node.unlockedBalance = payload.unlockedBalance;
        node.claimedBalance = payload.claimedBalance;
        node.isInitialized = true;
        ...
    }
}

function startRewardDistribution() external onlyOwner whenNotInitialized whenNotMigrated {
    isInitialized = true;
    emit RewardDistributionStarted();
}
```

Recommendation

Add a per-node guard before writing:

```
NodeData storage node = nodes[payload.nftId];  
if (node.isInitialized) revert NodeAlreadyInitialized(payload.nftId);
```

Update

• **Jun 19, 2026:** The issue has been acknowledged by the development team. They explained that this is an intentional operational design.

3.7. **LOW** - Burn For Migration Does Not Require Contract to Be Paused - **FIXED**

Position

- **Bullbit-Staking-Smart-Contract/contracts/IncentiveStakingVault.sol**
- **Bullbit-Staking-Smart-Contract/contracts/IncentiveRewardVault.sol**
- **Bullbit-Node-Reward-Smart-Contract/contracts/NodeRewardVault.sol**

Description

The audit brief specifies that `pause()` must be called across all vaults before `burnForMigration()` is executed: "The GovernanceManager calls the global `pause()` hook across all vaults, rendering them static". None of the three contracts enforce this sequence on-chain. `burnForMigration()` contains no paused-state guard and can be called on a fully live, unpaused vault.

This creates a mismatch between the documented migration process and the enforceable contract behavior. If the owner triggers migration before pausing all vaults, the burn can execute while user-facing state transitions are still enabled on other contracts or earlier in the same operational sequence.

```
// IncentiveStakingVault.sol  
function burnForMigration() external {  
    _checkOwner();  
    _requireNotMigrated();  
    if (address(bubiToken) == address(0)) revert BubiTokenNotSet();  
    updateGlobalIndex();  
    ...  
}  
  
// IncentiveRewardVault.sol  
function burnForMigration() external {  
    _checkOwner();  
    _requireNotMigrated();  
    if (address(bubiToken) == address(0)) revert BubiTokenNotSet();  
    ...  
}  
  
// NodeRewardVault.sol  
function burnForMigration() external onlyOwner whenInitialized whenNotMigrated {  
    uint256 burnAmount = bubiToken.balanceOf(address(this));  
    ...  
}
```

Recommendation

If the documented migration sequence is required for safety, add a paused-state guard to each `burnForMigration()` implementation. If unpaused migration is intentional, update the audit brief to state that pausing is an operational step rather than an enforced precondition.

Update

• **Jun 19, 2026:** The issue has been acknowledged and fixed by the development team. Both functions in `IncentiveStakingVault.sol` and `IncentiveRewardVault.sol` was updated to require `paused` state before executing.

3.8. LOW - Tier Lock Duration Has No Minimum or Maximum Bound - ACKED

Position

• `Bullbit-Staking-Smart-Contract/contracts/IncentiveStakingVault.sol`

Description

The audit brief states that active fixed staking tiers should have lock durations ranging from 3 to 36 months. The contract enforces only `duration != 0`. There are no `MIN_LOCK_DURATION` or `MAX_LOCK_DURATION` constants and no validation against the documented range.

The governance owner can configure a tier with a 1-second lock or a 100-year lock, and both values are accepted without revert. A 1-second duration negates the intended lockup model. A duration exceeding the documented maximum can trap user funds far beyond the expected staking term.

```
function updateTierConfig(uint256 tierId, uint256 duration, uint256 kValue, uint256 hValue) external {
    _checkOwner();
    _requireNotMigrated();
    if (tierId == 0 || tierId > MAX_TIER_ID) revert InvalidTier(tierId);

    if (_tierExists(tierId)) {
        updateGlobalIndex();
    }
    _setTierConfig(tierId, duration, kValue, hValue);
}

function _setTierConfig(uint256 tierId, uint256 duration, uint256 kValue, uint256 hValue) internal {
    if (tierId == 0 || tierId > MAX_TIER_ID) revert InvalidTier(tierId);
    if (duration == 0 || kValue == 0) revert InvalidTierConfig(tierId);

    tiers[tierId] = TierInfo({durationInSeconds: duration, kValue: kValue, hValue: hValue});
    emit TierConfigUpdated(tierId, duration, kValue, hValue);
}
```

Recommendation

Introduce named minimum and maximum lock duration constants and enforce them in `_setTierConfig()`. If the documented 3-to-36-month range is not intended to be a hard invariant, update the audit brief to describe the actual governance-controlled parameter range.

Update

- **Jun 19, 2026:** The issue has been acknowledged by the development team. They decided to maintain the current implementation without adding hardcoded duration constraints on-chain to preserve parameter flexibility for future staking campaigns. All parameter updates will be gated behind their multi-sig governance framework.

3.9. **LOW** - Unbonding Duration Has No Minimum Bound - **FIXED**

Position

- `Bullbit-Staking-Smart-Contract/contracts/IncentiveStakingVault.sol`

Description

The audit brief designates the 21-day unbonding window as a safety mechanism to prevent bank runs, and states it remains strictly enforced even during emergency unlock. In the contract, the only setter guard is `newDuration != 0`, so the governance owner can set `unbondingDuration = 1` and make the protection effectively absent.

Because `unbondingDuration` is read at `withdraw()` time rather than stored when unbonding starts, reducing the global duration can instantly make all in-progress unbonding positions withdrawable.

```
function setUnbondingDuration(uint256 newDuration) external {
    _checkOwner();
    if (newDuration == 0) revert ZeroAmount();

    uint256 previousDuration = unbondingDuration;
    unbondingDuration = newDuration;
    emit UnbondingDurationUpdated(previousDuration, newDuration);
}

function withdraw(uint256 lockId) external {
    ...
    uint256 withdrawableAt = record.unbondingStartTimestamp + unbondingDuration;
    if (block.timestamp < withdrawableAt) revert UnbondingPeriodNotMet(lockId,
        withdrawableAt);
    ...
}
```

Recommendation

Enforce a minimum value matching the documented invariant, such as `MIN_UNBONDING_DURATION = 21 days`. Also consider snapshotting the applicable unbonding duration per lock so future governance changes do not retroactively shorten or lengthen existing unbonding periods.

Update

- **Jun 19, 2026:** The issue has been acknowledged and addressed by the development team by replacing the minimum bound concept with an absolute maximum cap of 30 days. The team intentionally avoided imposing an on-chain minimum restriction to maintain strategic flexibility, allowing the governance framework to actively lower the unbonding period below 21 days during special campaign cycles for faster liquidity access. To still ensure proactive user protection against the systemic risk of indefinite locking of funds, the contract hardcodes a maximum cap of 30 days, mathematically guaranteeing that the protocol can never expand the unbonding window beyond this defensive threshold.

3.10. **LOW** - `getPendingReward()` Returns Stale Rewards - **FIXED**

Position

- `Bullbit-Staking-Smart-Contract/contracts/IncentiveStakingVault.sol`

Description

```
function getPendingReward(uint256 lockId) external view returns (uint256) {
    LockupRecord memory record = lockupRecords[lockId];
    if (record.userAddress == address(0)) return 0;

    return Math.mulDiv(record.amount, globalIndices[record.tierId] - record.entryIndex,
PRECISION);
}
```

`globalIndices[tierId]` reflects the last `updateGlobalIndex()` call, not the current block. The returned value represents rewards accrued up to `lastUpdateTimestamp`, consistently underreporting actual pending rewards. Off-chain integrations and user interfaces relying on this value will display incorrect balances.

The pending reward formula is duplicated in both `getPendingReward()` and `_claimReward()`, which can lead to inconsistencies if the formula is updated in one place but not the other. Create an internal function that computes pending rewards from a record, and call it from both functions to keep the logic consistent.

Recommendation

Compute the live index delta inline before returning, mirroring the logic in `updateGlobalIndex()`, or document the staleness explicitly in NatSpec so integrators know to call `updateGlobalIndex()` first.

Update

- **Jun 19, 2026:** The issue has been acknowledged and fixed by the development team.

3.11. **LOW** - `hValue` Can Be Set To Zero, Causing System-Wide Division by Zero Revert - **ACKED**

Position

- `Bullbit-Staking-Smart-Contract/contracts/IncentiveStakingVault.sol`

Description

In the `_setTierConfig` function, there are sanity checks to ensure `duration` and `kValue` are strictly greater than zero. However, `hValue` is allowed to be set to 0.

`hValue` acts as a crucial smoothing parameter and a safeguard in the denominator of the APR calculation inside `_computeIndexDeltas()`:

```
uint256 aprScaled = Math.mulDiv(tier.kValue, PRECISION, Math.sqrt(tier.hValue +
Math.mulDiv(totalStakedGlobal, 1, 10 ** 18)));
```

If governance configures a tier with `hValue = 0` and the vault only contains a "dust" stake (i.e., `totalStakedGlobal` is less than $1e18$), `Math.mulDiv(totalStakedGlobal, 1, 10 ^ 18)` truncates to 0. Consequently, `Math.sqrt(0 + 0)` evaluates to 0.

This results in a Division by Zero revert when calculating `aprScaled`, permanently bricking the `updateGlobalIndex()` function. Because `updateGlobalIndex()` is called at the beginning of stake, `initiateUnstake`, `claimReward`, and `withdraw`, the entire protocol becomes unusable, locking all funds and preventing further interactions.

Recommendation

Add `hValue` sanity check to `_setTierConfig` function to prevent setting `hValue` to 0.

Update

- **Jun 24, 2026:** The issue has been acknowledged by the development team. They decided to maintain the current implementation, confident that natural liquidity scaling provides sufficient protection.

3.12. INFO - Node License NFT Is Pausable Despite Being Documented as Non-Pausable - ACKED

Position

- `Bullbit-Node-Reward-Smart-Contract/contracts/NodeLicenseNFT.sol`

Description

The audit brief states that `NodeLicenseNFT` is "entirely non-upgradeable and non-pausable to assure node holders of permanent ledger immutability". The implementation is non-upgradeable, but it is not non-pausable: it inherits OpenZeppelin `Pausable` and exposes `pause()` and `unpause()` entrypoints.

This creates a mismatch between the documented immutability guarantee and the actual operational controls. While pausing does not allow NFT transfers or alter existing ownership, it can stop admin minting and changes the contract's live operating state. It also means the admin role is not solely limited to minting, because `onlyAdminOrOwner` can call `pause()`.

```
contract NodeLicenseNFT is ERC721, Ownable2Step, Pausable, IAdminable, IGovernable {
    ...
    function pause() external onlyAdminOrOwner {
```

```
        _pause();
    }

    function unpause() external onlyOwner {
        _unpause();
    }

    function mint(address to, uint256 tokenId) external onlyAdmin whenNotPaused {
        _mintNode(to, tokenId);
    }

    function batchMint(address[] calldata recipients, uint256[] calldata tokenIds)
    external onlyAdmin whenNotPaused {
        ...
    }
}
```

Recommendation

Decide whether pausing node licenses is intended. If the intended guarantee is truly non-pausability, remove **Pausable**, **pause()**, **unpause()**, and the **whenNotPaused** mint guards. If emergency pause is intentional, update the audit brief to describe the admin/owner pause authority and narrow the immutability claim to ownership transfer restrictions.

Update

- **Jun 19, 2026:** The issue has been acknowledged by the development team. The team decide keep the current implementation as a critical defensive control.

4. Version History

Version	Date	Status/Change	Created by
1.0	<i>Jun 19, 2026</i>	Private Report	Verichains
1.1	<i>Jun 24, 2026</i>	Public Report	Verichains

Table 2: Report versions history